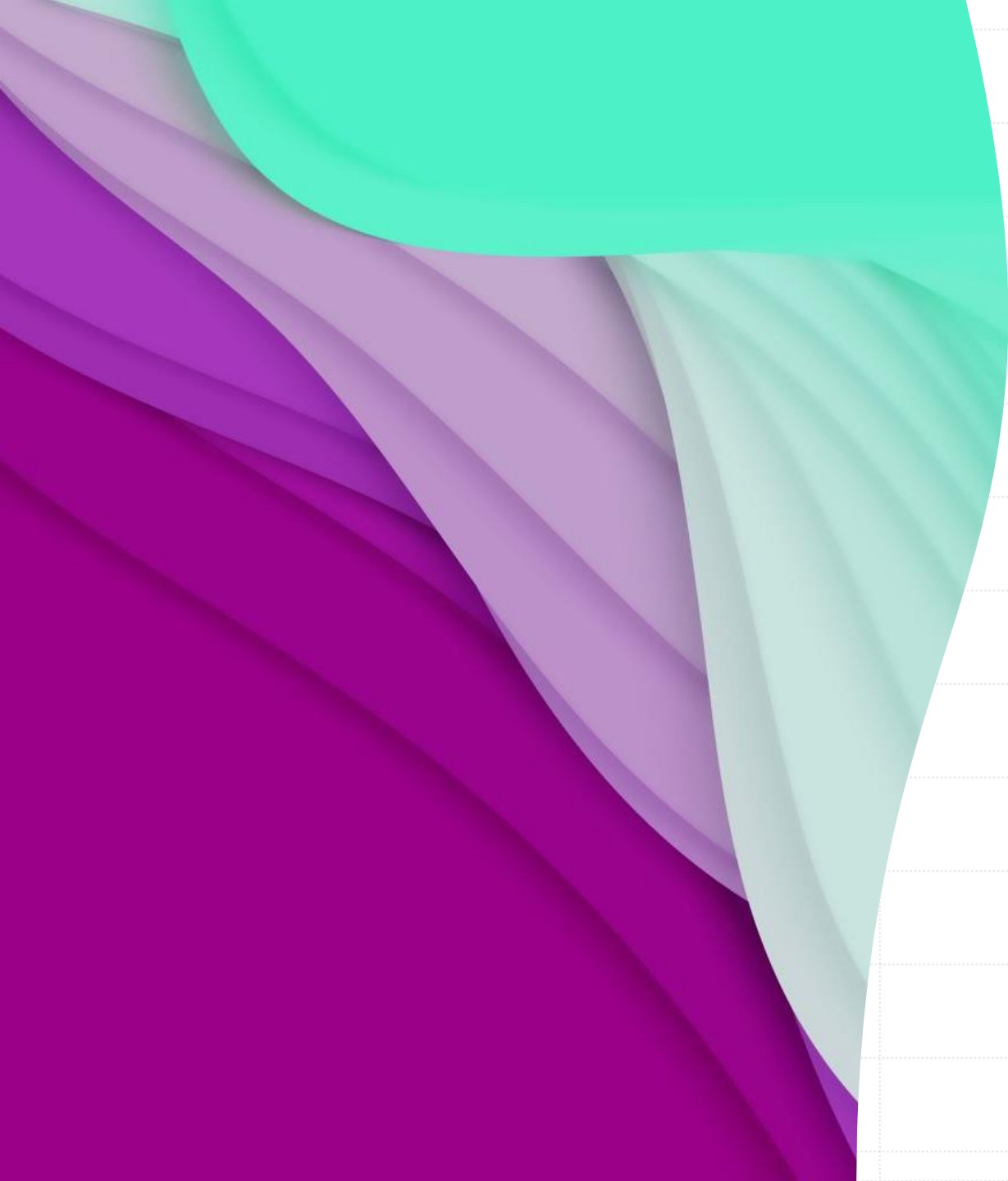




Python Programming Bootcamp – Week 3, Day 3

Tuples and Dictionaries – Introduction and
Operations

Atta Muhammad

attapanhyar@quest.edu.pk

+923331971110

Quaid-e-Awam University of Engineering,
Science and Technology



Learning Objectives

- Understand tuples and their immutability.
- Learn basic operations on tuples and their use cases.
- Understand dictionaries, key-value pairs, and operations on dictionaries.
- Explore practical applications of tuples and dictionaries in Python programming.



Introduction to Tuples

- **Definition:** A tuple is an immutable, ordered sequence of items.
- **Example:**

```
fruits = ('apple', 'banana', 'cherry')  
print(fruits)
```



Accessing Tuple Elements

- **Indexing:** Tuples use zero-based indexing, similar to lists.

```
fruits = ('apple', 'banana', 'cherry')  
print(fruits)  
print(fruits[1]) # Output: banana
```



Tuple Unpacking

- **Definition:** You can assign values from a tuple to variables in a single line.

```
fruits = ('apple', 'banana', 'cherry')  
fruit1, fruit2, fruit3 = fruits  
print(fruit1)  # Output: apple
```



Tuples are Immutable

- **Explanation:** Once a tuple is created, you cannot modify its elements.

```
fruits = ('apple', 'banana', 'cherry')  
fruits[0] = 'orange' # This will raise an error
```



When to Use Tuples

- **Use Cases:**

- Use tuples when you need an ordered collection that should not be changed.
- Tuples are often used to return multiple values from a function.

Hands-On Exercise 1

- **Task:** Create a tuple with the names of 5 cities and print the third city.





Introduction to Dictionaries

- **Definition:** A dictionary is a collection of key-value pairs, where each key is unique.

SYNATX:

```
my_dict = {key1: value1, key2: value2}
```

EXAMPLE

```
student = {'name': 'John', 'age': 20, 'major': 'Computer Science'}
```



Accessing Dictionary Values

- Using Keys:

```
student = {'name': 'John', 'age': 20, 'major': 'Computer Science'}  
print(student['name']) # Output: John
```



Modifying a Dictionary

- Adding/Updating Values

```
student = {'name': 'John', 'age': 20, 'major': 'Computer Science'}  
student['age'] = 21 # Update age  
student['gpa'] = 3.8 # Add new key-value pair
```



Removing Items from a Dictionary

- Using **del** and **pop()**:

```
student = {'name': 'John', 'age': 20, 'major': 'Computer Science'}  
del student['major'] # Remove key 'major'  
major = student.pop('gpa') # Remove and return 'gpa'
```



Looping Through a Dictionary

- Keys, Values, and Items:

```
student = {'name': 'John', 'age': 20, 'major': 'Computer Science'}  
for key, value in student.items():  
    print(f'{key}: {value}')
```

Hands-On Exercise 2

- **Task:** Write a Python program to create a dictionary with details of a book (title, author, year). Add a new key for the genre and print all the details.





Dictionary Methods

- **Common Methods:**
 - `get()`, `update()`, `keys()`, `values()`, `items()`

```
student = {'name': 'John', 'age': 20, 'major': 'Computer Science'}  
print(student.get('name', 'Unknown')) # Output: John
```



Hands-On Group Activity

- **Task:** Create a dictionary representing a classroom. The keys are the student names, and the values are their grades. Use loops to calculate the average grade.



Recap and Key Takeaways

- **Key Points:**

- Tuples are immutable sequences of items, while dictionaries are mutable collections of key-value pairs.
- Tuples are ideal for ordered, fixed data, while dictionaries are used for fast lookups and organizing data by keys.



Q&A and Wrap-Up



MATCH THE SOLUTION 🙄



EXERCISE 🤪

Step 1: Create a tuple with 5 city names

```
cities = ('New York', 'London', 'Paris', 'Tokyo', 'Sydney')
```

Step 2: Access and print the third city (index 2 since tuples are zero-indexed)

```
print(cities[2]) # Output: Paris
```



EXERCISE 🤪

Step 1: Create a dictionary for a book

```
book = {  
    'title': 'The Great Gatsby',  
    'author': 'F. Scott Fitzgerald',  
    'year': 1925  
}
```

Step 2: Add a new key-value pair for the genre

```
book['genre'] = 'Fiction'
```

Step 3: Print all the details of the book

```
for key, value in book.items():  
    print(f'{key}: {value}')
```



EXERCISE



Step 1: Create a dictionary with student names as keys and grades as values

```
classroom = {  
    'Alice': 85,  
    'Bob': 92,  
    'Charlie': 78,  
    'Diana': 88,  
    'Eve': 95  
}
```

Step 2: Calculate the total sum of the grades

```
total_grades = sum(classroom.values())
```

Step 3: Calculate the average grade

```
average_grade = total_grades / len(classroom)
```

Step 4: Print the average grade

```
print(f'Average grade: {average_grade:.2f}')
```